

# Tema 2.2

## 1. Evaluación de cortocircuito y precedencia

```
console.log( || "Fallback"); // "Fallback"
console.log( ?? "Fallback"); // 0
console.log( + "5");         // "55"
let i = ;
console.log(i++ * ++i);      // 8
```

- `0 || "Fallback"` da `"Fallback"`: el `||` devuelve el primer operando truthy, y `0` es falsy, así que pasa al siguiente.
- `0 ?? "Fallback"` da `0`: la coalescencia nula solo salta si el operando es `null` o `undefined`; el `0` es un valor legítimo y se respeta.
- `5 + "5"` da `"55"`: con `+`, si uno de los operandos es texto, el otro se convierte a texto y se concatena.
- `i++ * ++i` da `8` con `i = 2`: `i++` devuelve `2` y deja `i` en `3`; después `++i` sube a `4` y devuelve `4`. Queda `2 * 4 = 8`, e `i` acaba valiendo `4`.

## 2. Verificación exhaustiva de variables y operadores

```
// 1. Ámbito de bloque e inmutabilidad
var variableGlobal = "Accesible fuera";
{
  let variableLocal = "Aislada en bloque";
  const CONFIG_APP = { version "1.0.0", entorno "test" };

  // Mutabilidad de propiedad en un objeto declarado con const
  CONFIG_APP.entorno = "produccion";
  console.log("Objeto modificado:", CONFIG_APP);
}

// 2. Comprobaciones con el operador typeof
console.log("typeof 50n:", typeof n);           // "bigint"
console.log("typeof Symbol():", typeof Symbol()); // "symbol"
console.log("typeof null:", typeof null);        // "object" (anomalía de 1995)
console.log("typeof NaN:", typeof NaN);          // "number"

// 3. Coerción y comparaciones
console.log("5 == '5':", == "5");               // true
console.log("5 === '5':", === "5");             // false
console.log("5 + '5':", + "5");                 // "55"
console.log("5 - '5':", - "5");                 // 0

// 4. Operadores especiales in, delete e instanceof
const alumno = { id , curso "2 DAW", activo true };
console.log("¿Tiene propiedad 'curso'?:", "curso" in alumno); // true
delete alumno.activo;
console.log("Objeto tras borrado:", alumno);           // {
  id: 1001, curso: '2 DAW' }
console.log("alumno es instancia de Object:", alumno instanceof Object); // true
```

Salida real por consola:

```
Objeto modificado { version '1.0.0', entorno 'produccion' }
typeof n bigint
typeof Symbol() symbol
typeof null object
typeof NaN number
== '5' true
=== '5' false
+ '5'
- '5'
Tiene propiedad 'curso'? true
Objeto tras borrado { id , curso '2 DAW' }
alumno es instancia de Object true
```

Claves: un objeto declarado con `const` no se puede reasignar, pero sus propiedades sí se modifican; `typeof null` devuelve "object" por un error histórico, y `typeof NaN` devuelve "number" aunque signifique Not-a-Number.

## 3. Validador de tipos

Modificación 1: operador AND y doble negación.

Añadido en laboratorio\_operadores.js:

```
const operacionAnd = valA && valB;  
const negacionA = !!valA;
```

Y dentro del visor.innerHTML:

```
<strong>Cortocircuito AND (A && B)</strong> ${String(operacionA  
nd)}<br>  
<strong>Doble negacin (!!A)</strong> ${negacionA}
```

El && devuelve el primer operando falsy o, si todo es verdad, el último. El !!valA convierte cualquier valor a su booleano primitivo ("", 0, null, undefined y NaN pasan a false).

Modificación 2: aviso de precisión con BigInt.

Añadido justo antes de pintar el visor:

```
let aviso = "";  
if (typeof valA === "number" && Math.abs(valA) > Number.MAX_SAFE_IN  
TEGER) {  
  aviso = "Aviso: Precisión comprometida. Utiliza BigInt (sufijo n)  
para operar en este rango";  
}
```

Y en la salida:

```
${aviso ? <strong style="color:#f44747">${aviso}</strong><br>  
""}
```

La comprobación exige typeof valA === "number" para no intentar comparar textos u objetos. Number.MAX\_SAFE\_INTEGER vale 9007199254740991: a partir de ahí el tipo number pierde dígitos y hay que usar 123n.

## 4. Análisis de precedencia y operadores unarios combinados

```
let x = ;  
let y = ;  
let resultado = x++ * ++y + (x & );
```

| Paso | Qué ocurre                            | Estado |
|------|---------------------------------------|--------|
| 1    | x++ devuelve 3 y después incrementa x | x = 4  |
| 2    | ++y incrementa y y devuelve 5         | y = 5  |
| 3    | x & 1 es 4 & 1 = 0100 & 0001 = 0      | x = 4  |
| 4    | 3 * 5 + 0 = 15 + 0                    | —      |

Resultado: resultado = 15, x = 4, y = 5.

## 5. Comparativa de cortocircuitos con valores falsy críticos

```
function fijarDescuento(descuento) {  
  let tasaFinalA = descuento || 15;  
  let tasaFinalB = descuento ?? 15;  
  console.log(Tasa A: ${tasaFinalA} | Tasa B: ${tasaFinalB});  
}  
  
fijarDescuento(0);
```

Imprime Tasa A: 15 | Tasa B: 0.

El 0 es un valor legítimo: significa 0 % de descuento. El || solo mira si el operando es truthy y, al ser 0, lo descarta y devuelve 15, informando de un descuento que no existe. El ?? solo salta ante null o undefined, así que respeta el 0.

La forma correcta es tasaFinalB = descuento ?? 15. El || solo sería válido si el parámetro fuese opcional y el 0 no pudiera distinguirse de "sin valor".

## 6. Ejercicio tipo test

1. b) undefined. var micoche; reserva la variable y el motor le asigna undefined hasta que se le ponga un valor; no es null.
2. c) "125". Con + y un operando texto, se concatena en lugar de sumar.
3. c) ===. Exige igualdad de valor y de tipo, sin conversiones ocultas.
4. c) "object". Anomalía histórica de 1995 que se conserva por compatibilidad.
5. c) ??=. Solo asigna si la variable es null o undefined, dejando intactos 0 y false.
6. b) notas.push(7). const impide reasignar la referencia y redeclarar, pero permite mutar el contenido del array.